03 — Initial System Blueprint

Task 3 — Sprint 1 — Foundation & Planning

Blueprint level: *what the parts are and how information moves*. Implementation-level contracts are in `10_Detailed_Solution_Design.md`; diagrams in `06_High_Level_Architecture.md`.

---

1. Layered View

The project README mandates six layers. This blueprint keeps them and adds two cross-cutting layers (Safety, Observability) that the SLOs require.

#	Layer	Responsibility	Key components
L1	Query Interface	Accept analyst queries, authenticate, attach role + portfolio scope, return the response envelope	FastAPI `/api/v1/query`, request/response schemas, auth dependency, session id
L2	Query Understanding	Normalize the query, classify category + risk level, extract entities (portfolio, asset, sector, region, threshold), decide the path	Router Agent, entity extractor, deterministic keyword pre-filter
L3	Agent Orchestration	Execute the chosen path as a state machine, fan out/in, retry, enforce budgets, terminate on escalation	LangGraph graph, shared `GraphState`, conditional edges, node timeouts
L4	Knowledge Retrieval	Semantic + keyword retrieval over the 7 policy PDFs with page/section metadata	pgvector store, hybrid retriever, reranker, chunk metadata
L5	Data Analysis	Read-only parameterized access to `finance_risk_db`; compute derived risk/allocation metrics	SQL template registry, executor, risk scorer, allocation/concentration calculator, benchmark comparator
L6	Response Synthesis	Assemble a grounded answer with citations, risk summary, and disclaimer	Synthesis Agent, citation formatter, output validator
X1	Safety & Governance *(cross-cutting)*	Input guardrails, escalation rules, output validation, RBAC scoping, refusal templates	Guardrail module, escalation rule engine (deterministic), RBAC filter, Escalation Agent
X2	Observability & Cost *(cross-cutting)*	Structured logs, traces, metrics, token/cost accounting, eval hooks	JSON logger, OpenTelemetry spans, `/metrics`, cost meter, eval harness

---

2. Component Inventory

2.1 Agents (7)

Agent	Type	Inputs	Outputs	Tools
Router	LLM (small/fast) + deterministic pre-filter	raw query, role, session context	`category`, `risk_level`, `entities`, `path`, `escalation_intent`	none
Retrieval	LLM-assisted retrieval	query, entities, category	ranked policy chunks with `{doc, section, page, score}`	vector search, keyword search, reranker
Market Data	Tool-driven	entities (region, indicator, date range)	indicator rows + trend interpretation	`sql.market_indicators.*` templates
Portfolio Evaluation	Tool-driven + LLM	entities (portfolio, sector, asset)	holdings, allocation %, concentration, benchmark, decision history	`sql.portfolios.*`, `sql.holdings.*`, `sql.decisions.*` templates
Risk	LLM + deterministic scorer	risk-log rows, policy thresholds, market context	`risk_level`, breached thresholds, `escalation_flag` roll-up, rationale	`sql.risk_logs.*` templates, risk classifier
Synthesis	LLM	evidence bundle (all of the above)	final answer + citations + contradiction notes + disclaimer	none
Escalation	Deterministic + LLM summarizer	trigger id, evidence bundle, query	human-handoff envelope (reason, context, recommended reviewer role)	none

2.2 Knowledge sources

Source	Type	Contents	Access
`01_Investment_Strategy_Portfolio_Allocation_Guide.pdf`	policy	asset allocation, diversification, sector rules, risk-adjusted return targets, rebalancing	L4 vector
`02_Risk_Management_Exposure_Policy.pdf`	policy	market/credit/liquidity risk definitions, **exposure limits**, stress-testing rules	L4 vector
`03_Regulatory_Compliance_Financial_Governance_Policy.pdf`	policy	compliance requirements, disclosure obligations, risk reporting, governance controls, **escalation procedures**	L4 vector
`04_Market_Analysis_Investment_Research_Guide.pdf`	method	trend identification, economic indicator interpretation, sector evaluation, macro impact	L4 vector
`05_Portfolio_Monitoring_Performance_Evaluation_Guide.pdf`	method	performance metrics, benchmark comparison, volatility, drawdown, attribution	L4 vector

Source	Type	Contents	Access
`06_Basel_III_Risk_Management_Summary.pdf`	regulation	capital adequacy, RWA, market risk, **liquidity coverage**	L4 vector
`07_Investment_Risk_Assessment_Framework.pdf`	framework	risk scoring methodology, classification tiers, **approval thresholds, escalation triggers**	L4 vector
`portfolios`	SQL	20 rows — portfolio_id, name, manager, risk_category, total_value, benchmark_index	L5 read-only
`portfolio_holdings`	SQL	120 rows — holding_id, portfolio_id, asset_name, asset_type, sector, allocation_percentage, market_value, last_updated	L5 read-only
`asset_risk_logs`	SQL	150 rows — risk_id, asset_name, risk_type, risk_score (0–100), risk_level, risk_description, assessment_date, **escalation_flag**	L5 read-only
`market_indicators`	SQL	80 rows — indicator_id, indicator_name, region, indicator_value, measurement_date, trend_direction	L5 read-only
`investment_decisions`	SQL	100 rows — decision_id, portfolio_id, asset_name, decision_type, decision_reason, risk_assessment_reference → `asset_risk_logs.risk_id`, decision_date, **approval_status**	L5 read-only

2.3 Stores

Store	Technology	Holds
Vector index	PostgreSQL + pgvector, table `policy_chunks`	chunk text, embedding, `doc_id`, `doc_title`, `section`, page, `chunk_index`, `doc_category`
Relational	PostgreSQL `finance_risk_db`	the 5 business tables
Session memory	Postgres table `sessions` / `turns` (short-term); in-process cache	last N turns, resolved entities, last portfolio context
Eval + audit	`eval/reports/`, `audit_log` table	eval runs, per-request decision records, escalations

3. Data Flows

3.1 Ingestion flow (offline, Sprint 4)

```
7 PDFs → extract (per page, keep page no.) → detect section headings → chunk (~800 tokens, 120
overlap,
never split a limits table) → embed → upsert into policy_chunks with {doc_id, doc_title, section,
page}
? validate: retrieval smoke test over 20 known-answer probes
```

```
generate_capstone_sql_data.py (seeded) → INSERT into finance_risk_db (5 tables)
? extend with targeted rows required by golden queries → assert data preconditions
```

3.2 Query flow (online) — the four paths

```
                       ?? C1 RAG      → Retrieval ——
Request → Auth/RBAC →  ?? C2 SQL      → Portfolio Eval / Market   ?? Synthesis → Validate →
Response
  Guardrail → Router → ?? C3 Hybrid   → Retrieval → SQL → Risk ——
                       ?? C4 HighRisk → Retrieval → SQL → Market  ?
                                        → Risk → Portfolio Eval ??
                       C5 Escalation → Escalation node (terminal, no answer)
```

Escalation can also be entered **from any path** when a deterministic trigger fires on the evidence.

3.3 Evidence bundle (the currency of the system)

Every agent contributes typed evidence; Synthesis may only use what is in the bundle.

```
EvidenceBundle
—— doc_evidence[]   {doc_id, doc_title, section, page, chunk_text, score}
—— sql_evidence[]   {query_id, tables[], sql_template, params, row_count, rows}
—— risk_evidence[]  {asset, risk_type, risk_score, risk_level, escalation_flag,
breached_thresholds[]}
—— market_evidence[]{indicator, region, value, date, trend_direction, interpretation}
—— conflicts[]      {policy_claim, data_observation, severity}
```

3.4 Response envelope (blueprint shape; schema in 14)

`answer` — `citations[]` — `routing {category, path, agents_used[]}` — `risk {level, escalation_flag, breached_thresholds[]}` — `escalation {required, trigger, reason, context}` — `confidence` — `disclaimer` — `trace_id` — `latency_ms` — `token_usage`

---

4. Agent Roles — collaboration rules

Rule	Statement
Single writer	Each agent writes only its own slice of `GraphState`; no agent overwrites another's evidence
Parallel where independent	Retrieval, Market Data, and Portfolio Evaluation fan out concurrently for C3/C4 — they have no data dependency
Risk is downstream of data	The Risk agent runs only after SQL/market evidence exists; it never guesses a risk score
Synthesis is last and blind to the DB	Synthesis has no tools; it cannot fetch, only compose from the bundle
Escalation pre-empts everything	If any trigger fires, the graph routes to Escalation and no answer is synthesized
Guardrails are pre- and post-	Input guardrail before Router; output validator after Synthesis (citations present, no advice, no leaked override method)
No agent-to-agent free chat	Agents communicate only through `GraphState` — deterministic, loggable, replayable

5. Blueprint-level Design Decisions (ratified as ADRs in `07`)

#	Decision	Rationale
B1	LangGraph state machine, not a free-form agent loop	Deterministic paths are testable and give measurable routing accuracy
B2	pgvector inside the same Postgres as the business data	One store, one connection pool, joinable metadata; no second service to run
B3	Named parameterized SQL templates, not text-to-SQL	Directly serves SQL Correctness >=95% and read-only safety
B4	Deterministic escalation rules, LLM only for the handoff summary	Escalation Recall = 1.0 is unachievable with a probabilistic gate
B5	Two-tier latency budget (fast paths <=2s, high-risk paths documented separately)	Honest engineering against a real conflict (see R1 in `01`)
B6	Citation is a first-class typed object, not prose	Enables automated Source Attribution Rate = 1.0 measurement
B7	Frontend deferred to Sprint 5+ behind a frozen API contract	Backend/eval velocity in Sprints 4–6

6. Blueprint Risks Carried Forward

R1 latency vs. depth — R2 hallucinated limits — R3 escalation miss — R5 SQL correctness — R7 hybrid contradictions — R12 PDF table extraction. See `01_Project_Approach.md` —5.

> **Disclaimer.** Synthetic/educational data. Not investment advice.